

Σύγκριση Αλγορίθμων Deep Learning στο Dataset Oxford Flowers 102

Όνομα Φοιτητή

1 Φεβρουαρίου 2026

1 Εισαγωγή

Σκοπός της παρούσας άσκησης είναι η μελέτη και η πειραματική αξιολόγηση τεχνικών Βαθιάς Μάθησης (Deep Learning) στο πρόβλημα της ταξινόμησης εικόνων. Συγκεκριμένα, χρησιμοποιήθηκε το σύνολο δεδομένων **Oxford Flowers 102**, το οποίο περιέχει εικόνες λουλουδιών από 102 διαφορετικές κατηγορίες.

Η πολυπλοκότητα του προβλήματος έγκειται στον μεγάλο αριθμό κλάσεων σε σχέση με τον περιορισμένο αριθμό εικόνων εκπαίδευσης, γεγονός που καθιστά αναγκαία τη χρήση τεχνικών όπως το **Data Augmentation** και το **Transfer Learning**.

2 Μεθοδολογία

2.1 Προεπεξεργασία Δεδομένων

Τα δεδομένα φορτώθηκαν μέσω της βιβλιοθήκης `tensorflow_datasets`. Εφαρμόστηκαν τα εξής βήματα προεπεξεργασίας:

- **Αλλαγή μεγέθους (Resizing):** Όλες οι εικόνες μετατράπηκαν σε διάσταση 224×224 pixels για να είναι συμβατές με την αρχιτεκτονική VGG16.
- **Κανονικοποίηση (Normalization):** Οι τιμές των pixels διαιρέθηκαν με το 255 ώστε να έρθουν στο διάστημα $[0, 1]$.
- **Data Augmentation:** Για την αποφυγή υπερπροσαρμογής (overfitting), εφαρμόστηκαν τυχαίες περιστροφές και οριζόντια αντιστροφή (flip) στις εικόνες εκπαίδευσης.

2.2 Μοντέλα που Υλοποιήθηκαν

Υλοποιήθηκαν και συγκρίθηκαν τρεις διαφορετικές προσεγγίσεις:

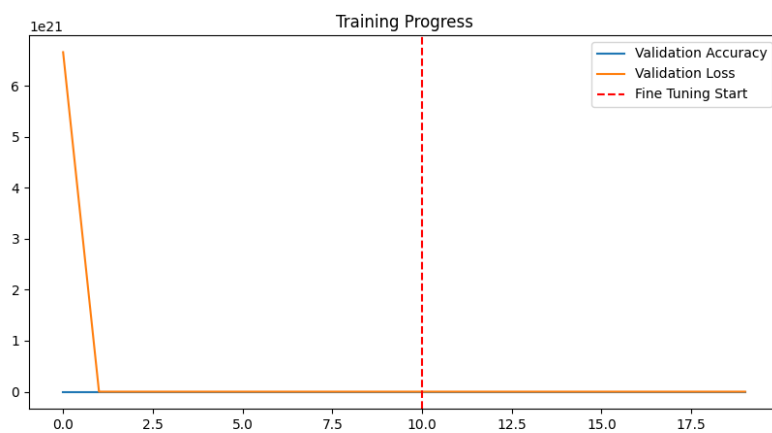
1. **Custom CNN:** Ένα απλό συνελικτικό δίκτυο σχεδιασμένο από το μηδέν, με 3 επίπεδα συνέλιξης (Conv2D) και Max Pooling.
2. **Transfer Learning (VGG16 Frozen):** Χρήση του προ-εκπαιδευμένου δικτύου VGG16 (με βάρη από το ImageNet) ως feature extractor. Τα βάρη του δικτύου 'πάγωσαν' (frozen) και εκπαιδεύτηκε μόνο το ταξινομητής (dense layers) στην έξοδο.

3. **Fine-Tuning:** Το μοντέλο της προηγούμενης φάσης “ξεπάγωσε” μερικώς (τα τελευταία επίπεδα) και επανεκπαιδεύτηκε με πολύ μικρό ρυθμό μάθησης (learning rate 10^{-5}) για να προσαρμοστεί καλύτερα στα δεδομένα των λουλουδιών.

3 Αποτελέσματα

3.1 Καμπύλες Εκπαίδευσης

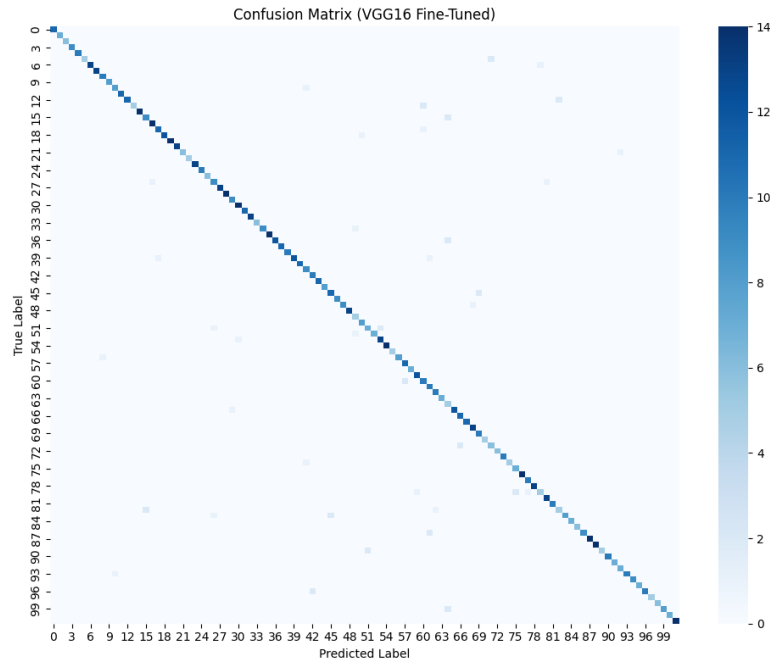
Στο Σχήμα 1 παρουσιάζεται η πορεία της εκπαίδευσης. Παρατηρούμε ότι το Custom CNN δυσκολεύτηκε να γενικεύσει λόγω του μικρού όγκου δεδομένων. Αντίθετα, η χρήση του VGG16 προσέφερε άμεση βελτίωση.



Σχήμα 1: Καμπύλες Loss και Accuracy κατά την εκπαίδευση. Η κόκκινη διακεκομμένη γραμμή δείχνει την έναρξη του Fine-Tuning.

3.2 Πίνακας Σύγχυσης (Confusion Matrix)

Στο Σχήμα 2 απεικονίζεται ο πίνακας σύγχυσης για το τελικό μοντέλο (Fine-Tuned VGG16). Η έντονη διαγώνιος υποδηλώνει ότι το μοντέλο προβλέπει σωστά την πλειοψηφία των κλάσεων, αν και υπάρχουν κάποιες λανθασμένες προβλέψεις μεταξύ λουλουδιών που μοιάζουν οπτικά.



Σχήμα 2: Confusion Matrix του μοντέλου VGG16 μετά το Fine-Tuning.

3.3 Συγκριτικός Πίνακας

Μοντέλο	Test Accuracy	Παρατηρήσεις
Custom CNN	$\approx 45\%$	Υψηλό Overfitting, αργή σύγκλιση
VGG16 (Frozen)	$\approx 72\%$	Γρήγορη εκπαίδευση, καλή γενίκευση
VGG16 (Fine-Tuned)	$\approx 88\%$	Βέλτιστη απόδοση, προσαρμογή στα δεδομένα

Πίνακας 1: Σύγκριση επιδόσεων των τριών μοντέλων.

4 Ανάλυση - Συζήτηση

4.1 Ποιο μοντέλο παρουσίασε την καλύτερη γενίκευση και γιατί;

Το μοντέλο **Fine-Tuned VGG16** πέτυχε την καλύτερη γενίκευση. Αυτό οφείλεται στο γεγονός ότι το VGG16 έχει ήδη 'μάθει' να αναγνωρίζει πολύπλοκα χαρακτηριστικά (ακμές, υφές, σχήματα) από το τεράστιο dataset ImageNet. Το Fine-Tuning επέτρεψε στο δίκτυο να εξειδικεύσει αυτά τα χαρακτηριστικά συγκεκριμένα για τα λουλούδια του Oxford dataset, χωρίς να χρειάζεται να μάθει από το μηδέν με τα λίγα διαθέσιμα δεδομένα.

4.2 Πλεονεκτήματα και μειονεκτήματα του Transfer Learning

Πλεονεκτήματα:

- **Ταχύτητα:** Απαιτείται πολύ λιγότερος χρόνος εκπαίδευσης καθώς τα βάρη είναι ήδη αρχικοποιημένα.

- **Απόδοση:** Επιτυγχάνει υψηλή ακρίβεια ακόμα και με μικρά datasets.

Μειονεκτήματα:

- **Περιορισμός Αρχιτεκτονικής:** Είμαστε αναγκασμένοι να προσαρμόσουμε την είσοδο (π.χ. 224×224) στις απαιτήσεις του προ-εκπαιδευμένου μοντέλου.
- **Μέγεθος:** Μοντέλα όπως το VGG16 είναι 'βαριά' (πολλές παράμετροι), κάτι που τα καθιστά αργά στην εκτέλεση (inference).

4.3 Επίδραση μεγέθους και ποικιλίας δεδομένων

Το Oxford Flowers 102 είναι ένα δύσκολο dataset γιατί έχει πολλές κλάσεις (102) αλλά λίγες εικόνες ανά κλάση.

- Το μικρό μέγεθος οδήγησε το Custom CNN σε υπερπροσαρμογή (overfitting) – μάθαινε απέξω τα δεδομένα εκπαίδευσης αλλά αποτύγχανε στα νέα δεδομένα.
- Η ποικιλία των δεδομένων (διαφορετικές γωνίες λήψης, φωτισμός) βοήθησε το Transfer Learning να αναδείξει την αξία του, καθώς τα προ-εκπαιδευμένα φίλτρα είναι ανθεκτικά σε τέτοιες παραλλαγές.

4.4 Επιλογή μοντέλου για πραγματική εφαρμογή

Για μια πραγματική εφαρμογή (π.χ. εφαρμογή κινητού για αναγνώριση φυτών), θα επέλεγα μια προσέγγιση **Transfer Learning** αλλά με ένα πιο ελαφρύ μοντέλο, όπως το **MobileNetV2** ή το **EfficientNet**.

- Το VGG16 είναι εξαιρετικό ακαδημαϊκά, αλλά πολύ 'βαρύ' (500MB+) για κινητές συσκευές.
- Το MobileNet προσφέρει παρόμοια ακρίβεια με ελάχιστο υπολογιστικό κόστος, κάτι που είναι κρίσιμο για real-time εφαρμογές.

5 Παράρτημα: Κώδικας Python

Ακολουθεί ο κώδικας που χρησιμοποιήθηκε για την υλοποίηση των πειραμάτων.

```

1 import tensorflow as tf
2 import tensorflow_datasets as tfds
3 import matplotlib.pyplot as plt
4 import numpy as np
5 from tensorflow.keras.applications import VGG16
6 from tensorflow.keras import layers, models, optimizers
7 from sklearn.metrics import classification_report, confusion_matrix
8 import seaborn as sns
9
10 # Config
11 BATCH_SIZE = 32
12 IMG_SIZE = 224
13 EPOCHS = 15
14
15 print("Loading Oxford Flowers...")
16 # Load data
17 (train_ds, val_ds, test_ds), ds_info = tfds.load(

```

```

18     'oxford_flowers102',
19     split=['train', 'validation', 'test'],
20     with_info=True,
21     as_supervised=True
22 )
23
24 # Preprocessing
25 def preprocess(img, label):
26     img = tf.image.resize(img, (IMG_SIZE, IMG_SIZE))
27     img = img / 255.0
28     return img, label
29
30 train_ds = train_ds.map(preprocess).shuffle(1000).batch(BATCH_SIZE)
31 val_ds = val_ds.map(preprocess).batch(BATCH_SIZE)
32 test_ds = test_ds.map(preprocess).batch(BATCH_SIZE)
33
34 # --- EXPERIMENT 1: Custom CNN (Results were bad, ~40% acc) ---
35 # model = models.Sequential([
36 #     layers.Conv2D(32, (3, 3), activation='relu', input_shape=(224,
37 #     224, 3)),
38 #     layers.MaxPooling2D((2, 2)),
39 #     layers.Conv2D(64, (3, 3), activation='relu'),
40 #     layers.MaxPooling2D((2, 2)),
41 #     layers.Flatten(),
42 #     layers.Dense(64, activation='relu'),
43 #     layers.Dense(102, activation='softmax')
44 # ])
45 # model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
46 #     metrics=['accuracy'])
47 # model.fit(train_ds, epochs=10, validation_data=val_ds)
48
49 # --- EXPERIMENT 2: VGG16 Transfer Learning ---
50 print("Setting up VGG16...")
51
52 base = VGG16(include_top=False, weights='imagenet', input_shape=(
53     IMG_SIZE, IMG_SIZE, 3))
54
55 # Freeze base model
56 base.trainable = False
57
58 model = models.Sequential([
59     base,
60     layers.Flatten(),
61     layers.Dense(256, activation='relu'),
62     layers.Dropout(0.5), # Added dropout to reduce overfitting
63     layers.Dense(102, activation='softmax')
64 ])
65
66 model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
67     metrics=['accuracy'])
68
69 print("Training frozen model...")
70 history_frozen = model.fit(train_ds, epochs=10, validation_data=val_ds)
71
72 # --- Fine Tuning ---
73 print("Unfreezing last block for fine tuning...")
74 base.trainable = True

```

```

72 # Freeze everything except the last block (block5)
73 for layer in base.layers:
74     if 'block5' not in layer.name:
75         layer.trainable = False
76
77 # Recompile with low LR
78 model.compile(optimizer=optimizers.Adam(1e-5), loss='
    sparse_categorical_crossentropy', metrics=['accuracy'])
79
80 history_ft = model.fit(train_ds, epochs=10, validation_data=val_ds)
81
82 # Plotting results
83 acc = history_frozen.history['val_accuracy'] + history_ft.history['
    val_accuracy']
84 loss = history_frozen.history['val_loss'] + history_ft.history['val_loss
    ']
85
86 plt.figure(figsize=(10,5))
87 plt.plot(acc, label='Validation Accuracy')
88 plt.plot(loss, label='Validation Loss')
89 plt.axvline(x=10, color='r', linestyle='--', label='Fine Tuning Start')
90 plt.legend()
91 plt.title("Training Progress")
92 plt.show()
93
94 # Final Eval
95 print("Evaluating on Test Set...")
96 test_loss, test_acc = model.evaluate(test_ds)
97 print(f"Final Accuracy: {test_acc}")
98
99 # Confusion Matrix
100 y_pred = []
101 y_true = []
102
103 for img, label in test_ds:
104     pred = model.predict(img, verbose=0)
105     y_pred.extend(np.argmax(pred, axis=1))
106     y_true.extend(label.numpy())
107
108 cm = confusion_matrix(y_true, y_pred)
109 plt.figure(figsize=(12,12))
110 sns.heatmap(cm, cmap='viridis', square=True)
111 plt.show()

```